

CANN • ASCEND 950 • REGBASE

Vector Function高性能编程指南

在Ascend 950上用Vector Function榨干Vector算力

技术解读 • CANN 社区 | 面向算子开发者

<https://gitcode.com/cann/cann-samples>

CANN

Contents

01 第一章 Hello Vector Function

编程模型入门

02 第二章 SIMD VF 编程规范

写出正确的代码，发挥硬件的性能

03 第三章 SIMD VF 访存

Load / Store 模式选择

第一章

Hello Vector Function

在 Ascend 950 上用 Vector Function 榨干 Vector 算力

01

Vector Function 编程模型

从MemBase到RegBase

02

Hello VF

第一个 Vector Function 代码

03

VF的执行细节

乱序执行与指令并行

04

编程建议

在Ascend950上优先使用RegBase实现功能

Vector性能天花板

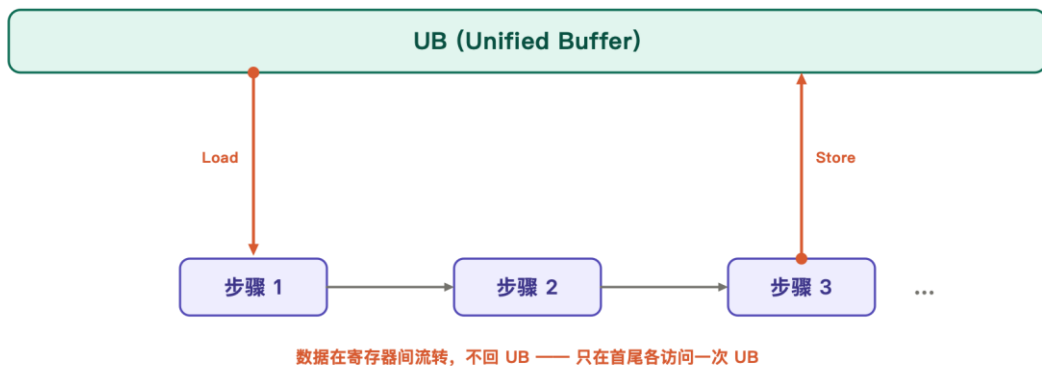
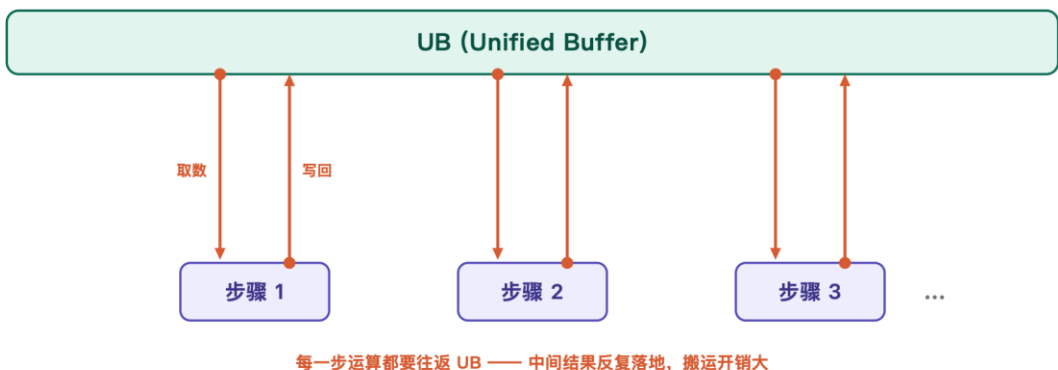
Profiling 的错觉：用 MemBase 写 GeLU，Vector 占用率 >95% 甚至 99%，Vector 计算单元满负荷。

访存瓶颈：占用耗时的不全是计算，相当一部分是数据在 UB 与 Reg 间搬进搬出。

MemBase 写法的开销：MemBase 让开发者面向 UB 编程，API 负责从指定的 UB 地址读写数据，开发者不感知寄存器内部细节。

MemBase 写法：Vector 占用率 > 95%，看似满负荷

但这段「忙碌」拆开看分三段——只有中间在真正计算



RegBase 让开发者面向寄存器编程，控制 Reg 处理细节，重新组织数据流，省掉数据频繁往返 UB 的搬运开销。

—— 一条指令作用在一整批元素 (SIMD)

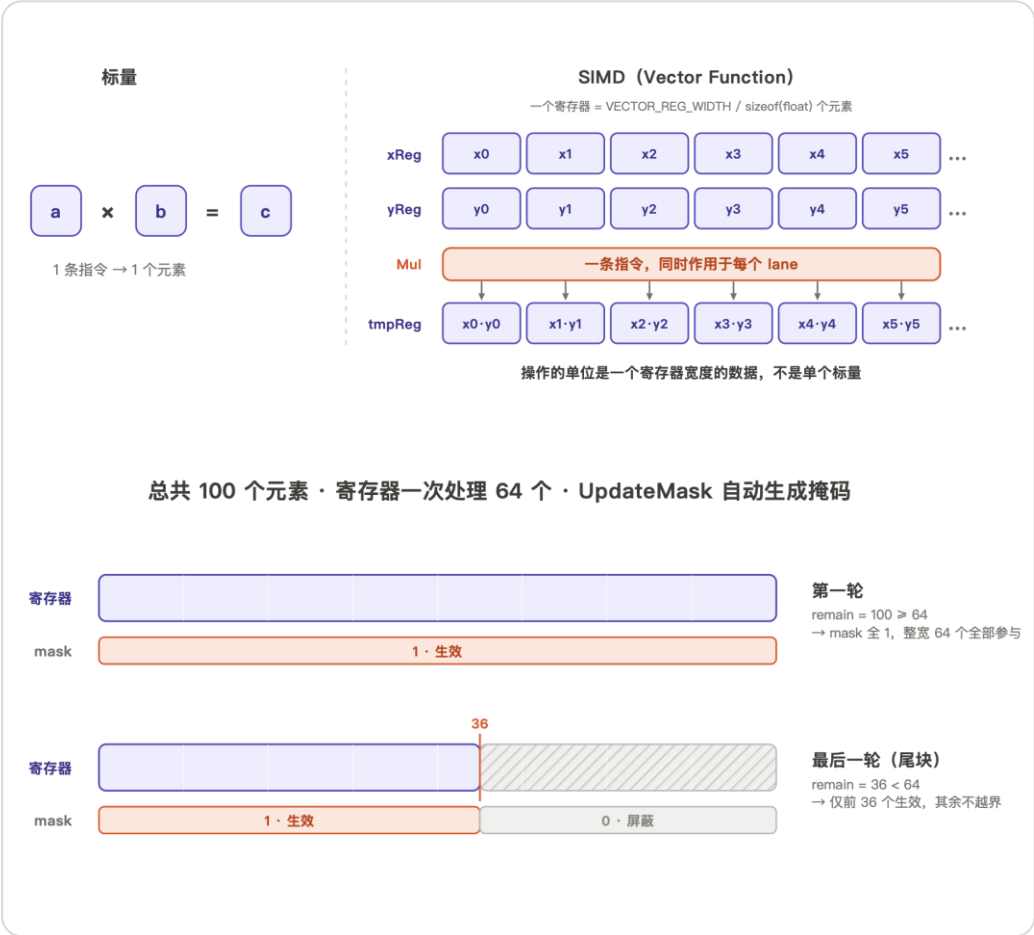
VF 的编程模型

—— `__simd_vf__` 是函数修饰符：告诉编译器这是 Vector Function，写法接近普通 C++——有输入输出参数、局部变量、多层循环。

—— 函数体用 `AscendC::Reg API`：操作寄存器的 Load、运算与 Store。

寄存器宽度是固定常量 `AscendC::GetVecLen()`，单位是Byte。
`Mul(tmp, x, y)` 不是算一对数，而是同时对一整批元素做乘法。
注意：操作单位是一个寄存器宽度的数据，不是单个标量。

—— **Mask 控制哪些元素生效**：一次处理一整批，凑不满一个寄存器宽度时怎么办？
例：总共 100 个 float，寄存器一次装 64——第一轮 64 个，第二轮只剩 36 个有效。
配置掩码寄存器：让运算只在有效元素上生效，其余不参与；尾块也能精确处理，不会越界。



动手: Hello VF

```
__simd_vf__ inline void muladd_vf(
    __ubuf__ float *xAddr,
    __ubuf__ float *yAddr,
    __ubuf__ float *zAddr,
    uint32_t total,
    uint32_t loopNum) {

    constexpr uint32_t vectorLength = AscendC::GetVecLen() / sizeof(float);
    AscendC::Reg::RegTensor<float> xReg, yReg, zReg, tmpReg;
    AscendC::Reg::MaskReg pMask;
    uint32_t remain = total;

    for (uint16_t i = 0; i < loopNum; ++i) {
        pMask = AscendC::Reg::UpdateMask<float>(remain);
        AscendC::Reg::LoadAlign(xReg, xAddr + i * vectorLength);
        AscendC::Reg::LoadAlign(yReg, yAddr + i * vectorLength);
        AscendC::Reg::LoadAlign(zReg, zAddr + i * vectorLength);
        AscendC::Reg::Mul(tmpReg, xReg, yReg, pMask);           // tmp = x * y
        AscendC::Reg::Add(zReg, tmpReg, zReg, pMask);           // z = tmp + z
        AscendC::Reg::StoreAlign<float, AscendC::Reg::StoreDist::DIST_NORM_B32>(
            zAddr + i * vectorLength, zReg, pMask);
    }
}
```

循环体内容

把数据从UB载入寄存器

使用Reg::Mul Reg::Add完成计算

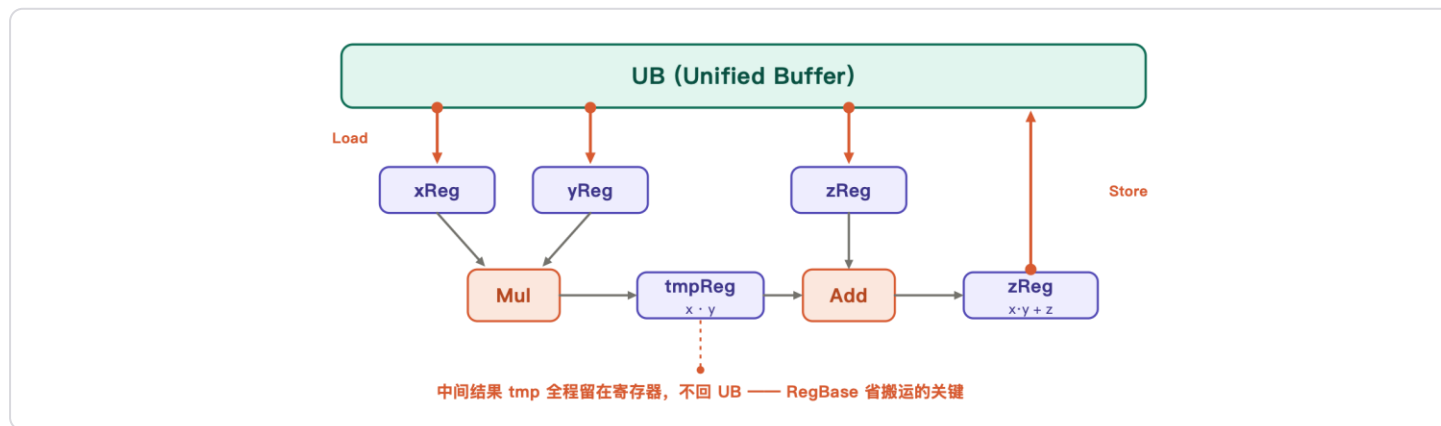
把数据写回UB

关键点

tmpReg 存放 x·y 的中间结果, 全程留在寄存器、不写回 UB——正是 RegBase 省搬运的关键。

动手：Hello VF

- 寄存器宽度** `vectorLength = AscendC::GetVecLen() / sizeof(float)` —— 一轮处理多少个 float。
- 寄存器变量** `RegTensor<float>` 表示存储float类型的Vector寄存器，每个变量对应一整批 float。
- Mask 掩码** `UpdateMask(remain)` 按 `remain` 生成掩码并就地吧 `remain` 减一个 `vectorLength`：前几轮掩码全开，最后一轮不足`vectorLength`个元素时掩码只让前 `remain` 个生效。
- 对齐要求** `LoadAlign / StoreAlign` 要求 UB 地址按 32B 对齐，否则触发 ERROR、Kernel 崩溃；需要访问非对齐UB地址时改用 `LoadUnalign / StoreUnalign`。



muladd_vf 的数据流：Load → Mul → Add → Store

VF 是怎么被调用的

```
__global__ __vector__ void pure_vf_kernel() {
    constexpr uint32_t kNumElements = 8192;
    constexpr uint32_t vectorLength =
        AscendC::GetVecLen() / sizeof(float);
    constexpr uint32_t total = kNumElements;
    uint32_t loopNum = (total + vectorLength - 1) / vectorLength;

    __ubuf__ float xLocal[kNumElements];
    __ubuf__ float yLocal[kNumElements];
    __ubuf__ float zLocal[kNumElements];

    asc_vf_call<muladd_vf>(xLocal, yLocal, zLocal, total, loopNum);
}
```

VF 名作模板参数；kernel 入口用 __vector__ 修饰，表示运行在 Vector Core 上。

loopNum 向上取整：总元素数 ÷ 一轮元素数，不能整除就多跑一轮，多出的那轮由 mask 控制有效范围。



kernel 与 VF 的调用关系

注：例子简化，省去真实算子的 GM ↔ UB 搬运。

执行时硬件在做什么

乱序执行：硬件不必按代码书写顺序逐条发射；一条指令只要操作数就绪、单元空闲就能提前发射。

为什么有用：每条指令都有执行延迟，让彼此无依赖的指令互相掩盖延迟，提高整体的利用率。

两层重叠：迭代内（两 Load、Exp/Sqrt 并行）+ 迭代间（下一轮 Load/Exp/Sqrt 提前填满空等周期）。

不同 VALU 的指令天然并行：比如一条 Mul 和一条 ReduceSum，占用不同硬件单元、互不阻塞，可同时执行。

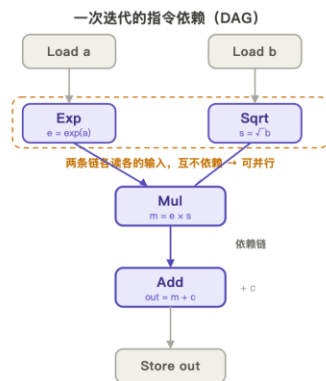
同类型两条也可并行：比如两条 Add 指令可以同时执行。

并非全部支持：两条 ReduceSum 无法同时执行，只能串行执行。

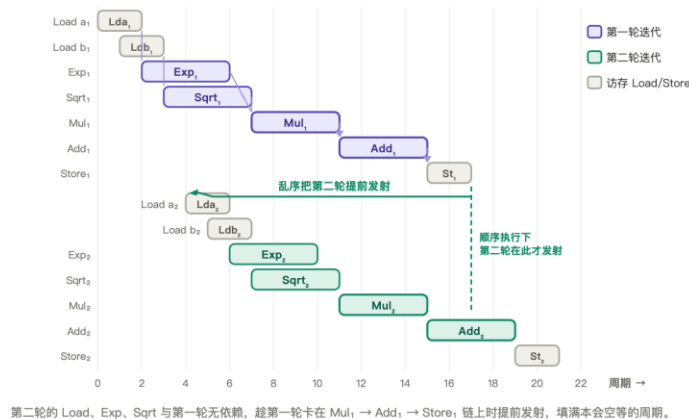
乱序的重要作用：在每个周期里尽可能凑出可并行的指令组合，把发射效率拉满。

乱序执行：无依赖的指令可以重叠，相邻迭代之间尤其明显

循环体 $out = exp(a) \times \sqrt{b} + c$ —— 先 Load a/b, Exp、Sqrt 互不依赖，Mul 等两者、Add 等 Mul，末尾 Store 回写



两轮迭代的执行流水（按周期）



指令并行执行

大部分相同指令可以并行执行，不同指令天然可以并行执行

相同指令

如 两条 Add



✓ 大部分可以并行执行

少数相同指令

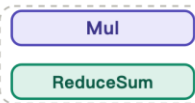
如 两条 ReduceSum



✗ 无法并行，只能串行

不同指令

如 Mul + ReduceSum



✓ 天然可以并行执行

执行时硬件在做什么

软件循环 · 有开销

- 每轮额外执行循环控制指令：计数器自增、与上界比较、条件分支跳回。
- 控制流开销不是有效计算，循环体越短，循环开销占比越高。
- 末尾条件分支引入数据依赖：打断流水，阻碍乱序与并行跨迭代展开。

硬件循环 · 低开销

- 一条 **VLOOP** 指令消除开销：计数器递增、边界比较、回跳全由硬件循环控制逻辑完成。
- 硬件预知迭代次数与边界：把相邻迭代纳入同一调度窗口，乱序 / 并行得以跨迭代生效。
- 不满足要求则退化(编译器实现)：退回带控制指令的软件循环。

正确性由硬件按寄存器编号判断依赖保证；但 **UB 的地址**需要开发者自己管理。

IPC = EXEC 指令数 ÷ Cycle 数

如何衡量执行效率

IPC = VF 中执行的 EXEC 类指令数 ÷ VF 消耗的 Cycle 数

分子= 只包括 EXEC 指令，不包含 Load / Store 访存指令。

- | **IPC 衡量什么：**计算单元的利用率，也就是乱序与并行的能力用了多少。
- | **比值越大越好（大多数情况下）：**每个周期塞进的计算指令越多，乱序与并行用得越充分。
- | **与“减少 UB 交互”的关系：**① 降低访存指令数量 ② IPC描述指令吞吐。
- | **减少了数据搬运但 IPC 低：**比如计算依赖链太长，凑不出可并行的指令组合。

IPC (Instructions Per Cycle, 每周期指令数)

IPC =

VF 中执行的 EXEC 类指令数

VF 消耗的 Cycle 数

EXEC 类 = 实际参与计算的指令，不含 Load / Store 访存指令

图 8 IPC：每周期塞进多少计算指令

编程建议

Membase 和 Regbase可以混合使用

建议使用Vector Function实现Vector Bound部分的计算

极致性能场景：完全使用Regbase编程

用 RegBase 精细控制寄存器，可以探索到 MemBase 达不到的性能区间。

建议：优先选择 RegBase 实现算子——自由度更高，性能天花板更高。

第二章

SIMD VF 编程规范

编写 SIMD VF 的四类规范，涵盖基本功能正确/基本性能有保证的写法

01

函数定义与调用 功能•性能

入参类型限制与数量建议、asc_vf_call 调用链的规则

02

硬件循环 Hardware Loop 性能

硬件循环的标准形态，和如何避免退化为软循环

03

LocalMemBar 同步机制 功能

VF 内具有隐式依赖的指令需要 LocalMemBar 机制保序

04

硬件资源/能力边界 功能•性能

寄存器 / 标量计算 / 地址计算 / 数据访问

函数定义与调用：入参类型限制

支持 • 参数形态

- `float scale / uint32_t n`: 基本算术类型
- `__ubuf__ T* x`: UB 上的地址
- `const __ubuf__ T* x`: UB 地址支持 `const` 修饰
- `__ubuf__ fp8_e5m2_t* x`: Reg API 支持的数据类型指针

暂不支持 • 参数形态

- `float a[4]`: 数组退化为指针, 不作为 VF 参数
- `float& s`: 引用的本质是指针
- `float* / __gm__*`: 非 UB 上的地址
- 函数指针 / 函数对象: 暂不支持
- 结构体传值: 暂不支持

// 正例: 只传 UB 地址和必要 scalar

```
template <typename T>
__simd_vf__ inline void SingleOpVF(__ubuf__ T* xUbAddr,
    __ubuf__ T* yUbAddr, T scalar,
    uint32_t elemNum, uint16_t vfLoopNum) {}
```

// 反例: 这些都不作为 SIMD VF 参数使用

```
__simd_vf__ inline void InvalidRefArg(float& s) {}
__simd_vf__ inline void InvalidPlainPtr(float* x) {}
__simd_vf__ inline void InvalidGmPtr(__gm__ float* x) {}
__simd_vf__ inline void InvalidArrayArg(float coeff[4]) {}
```

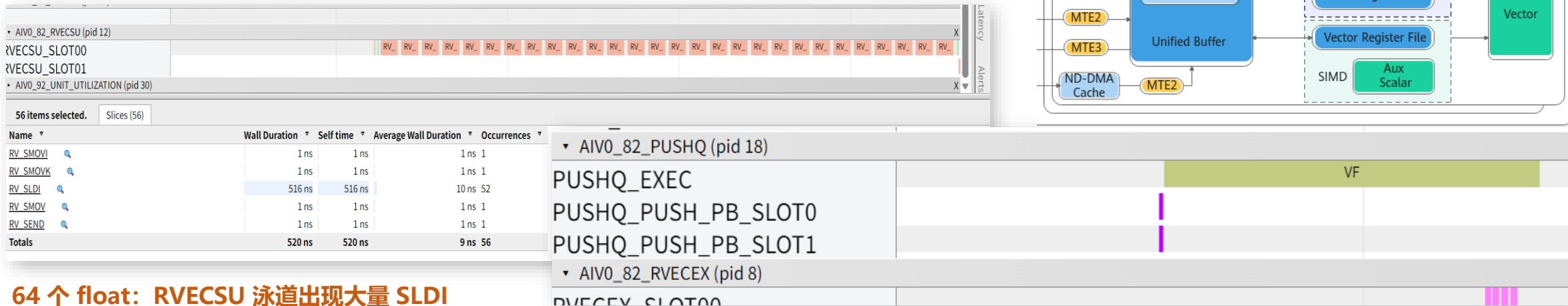
■ 入参过多会导致额外的开销

函数定义与调用：入参数量建议

VF 入参由 Scalar 单元写入 Parameter Buffer 后通过 PUSH_PB 指令传递给 Vector 单元内的标量寄存器。

风险：参数过多会放大 PB 传参开销，也可能引发 Vector 单元内标量寄存器溢出。

建议：只传 VF 必需参数，通常 28 个 32 位参数是安全的；编译期常量使用模板参数。



64 个 float: RVECSU 泳道出现大量 SLDI

8 个 float: PUSH PB 压力低, Vector 接收参数后直接启动 VF

// 正例：控制入参的数量

```
__simd_vf__ void ManyFloatArgs008VF( __ubuf__ float* x, __ubuf__ float* y,
    float p0, ... , float p7, uint32_t elemNum, uint16_t vfLoopNum);
```

asc_vf_call 是调用 **__simd_vf__** 的入口

函数定义与调用：asc_vf_call

asc_vf_call 可以调起 __simd_vf__ 函数。

__simd_vf__ 可以调用 __simd_callee__，编译时被内联。

```
// 正例 ✓
asc_vf_call<PlainVF>(x, y, scalar, n, loop);
asc_vf_call<TemplateVF<float,2>>(x, y, ...); // 显式实例化
__simd_callee__ void CalleeA(__ubuf__ float* x) {
    CalleeB(x); // callee 调 callee
}
__simd_vf__ void EntryVF(__ubuf__ float* x, ...) {
    CalleeA(x); // VF 调 callee, 内联展开
}
```

```
// 反例 X
asc_vf_call<TemplateVF>(...); // 未实例化模板名
asc_vf_call<SimdCalleeOnly>(...); // callee 不能作入口
EntryVF(...) { OtherVF(...); } // VF 调非 callee 子函数
CalleeA(...) { CalleeA(...); } // callee 递归调用
```

支持

- asc_vf_call<PlainVF>：普通 VF 作入口
- asc_vf_call<TemplateVF<float,2>>：模板 VF 显式实例化
- EntryVF(){ CalleeA(); }：VF 调用 callee
- CalleeA(){ CalleeB(); }：callee 调用 callee

不支持

- asc_vf_call<Callee>：callee 不能作入口
- asc_vf_call<TemplateVF>：不支持**自动推导**模板参数
- EntryVF(){ OtherVF(); }：VF 内不能调用另一个 VF
- Obj::MemberVF：成员函数不能作为 VF
- CalleeA(){ CalleeA(); }：callee 不支持递归调用，除非编译期能完全展开

标准循环头 + 稳定的数据流 = 优化为硬件循环

硬件循环 Hardware Loop

- 符合规范的 for 循环可生成 Hardware Loop 指令流（硬件循环）
- 不符合规范时，可能退化为由 Aux Scalar 控制指令构成的 Software Loop

判断项	推荐写法	说明
循环变量类型	uint16_t i	固定 16 位宽循环变量
头部三要素	i=0; i<vfLoopNum; ++i	起始 0、步长 1、边界外部传入
嵌套层数	≤ 4 层 for	每层都能对应硬循环
尾块依赖 mask 处理	mask = UpdateMask(elemNum)	自动递减 elemNum, 生成本轮 mask
尾块独立处理	for(0;i<hasTail;++i)	短 for 替代 if(hasTail)
编译期分支	if constexpr	编译期消除, 不进运行期
元素级条件	Reg::Compares + Reg::Select	表达为向量数据流

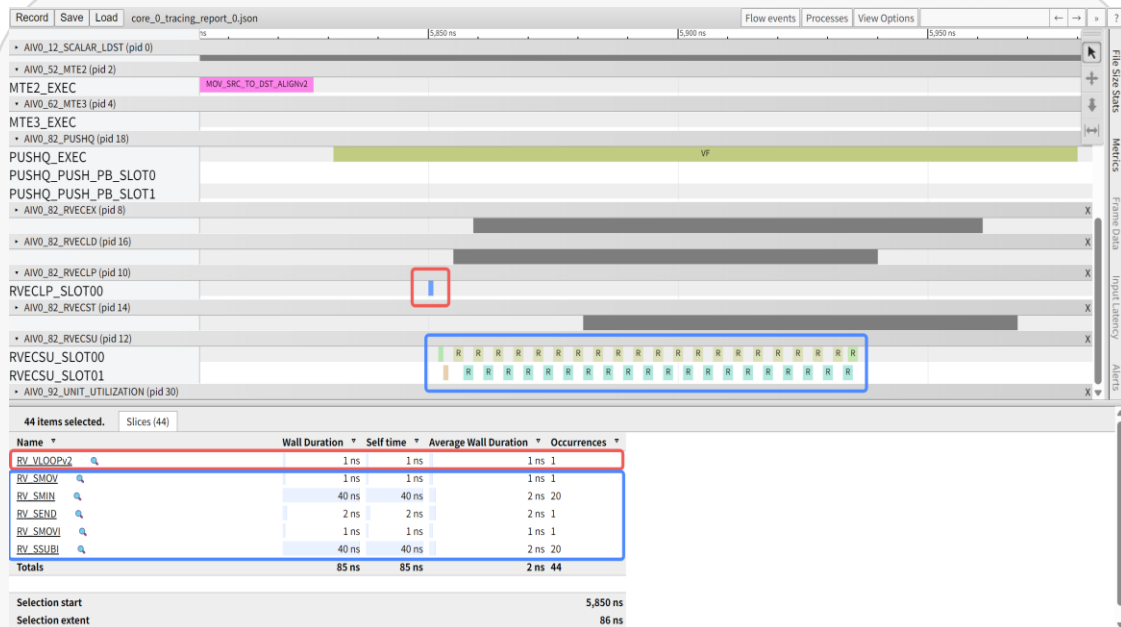
```
// 正例 ✓ 标准 for 头、线性地址、Mask 尾块
// VL_T = AscendC::GetVecLen() / sizeof(T)
for (uint16_t i = 0; i < vfLoopNum; ++i) {
    // 尾块交给 Mask。
    mask = Reg::UpdateMask<T>(elemNum);
    // 地址保持 base + i * VL_T。
    Reg::LoadAlign(xReg, x + i * VL_T);
    // 循环体保持直线数据流。
    Reg::StoreAlign(y + i * VL_T, yReg, mask);
}

// 反例 ✗ 运行期控制流
if (skip) continue/break;

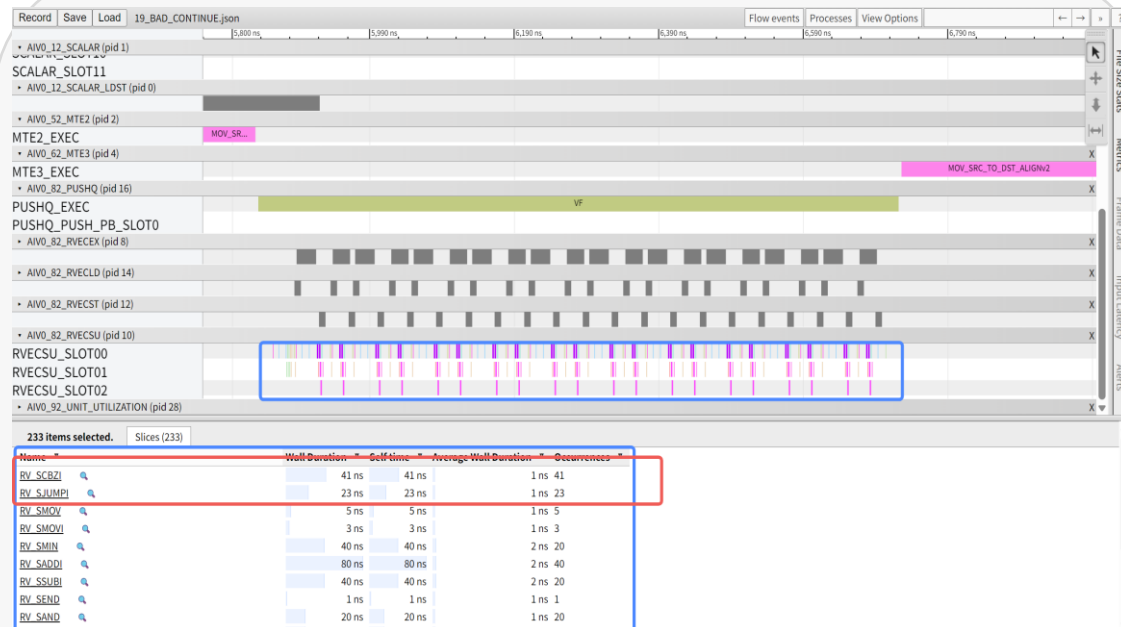
// 反例 ✗ 循环内修改边界或步长
for (i=0; i < bound; i += stride) {
    bound = ...; stride = ...;
}
```

通过仿真流水确认循环类型

示例：硬循环和软循环



标准硬循环：VLOOPv2 驱动



退化为软循环：RVECSU 出现控制流指令

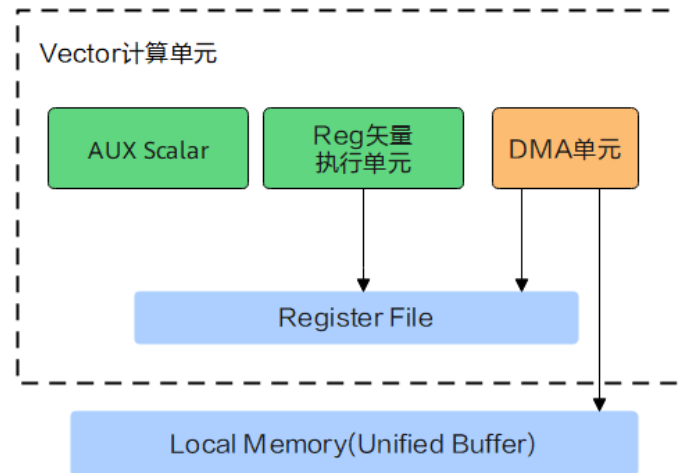
检查顺序：先确认是否存在 VLOOPv2，再查看 RVECSU 泳道是否出现 RV_SCBZI / RV_SJUMPI 等控制流指令。

Vector 单元内的指令保序机制

LocalMemBar 同步机制

Vector 单元的指令是乱序发射的，不同类别的指令分别由 Reg 矢量、DMA、Aux Scalar 三个子硬件单元各自执行，Reg 矢量执行单元和 DMA 单元的指令可以各自乱序执行指令。

对于隐式数据依赖的指令，使用 LocalMemBar<src, dst>() 可约束 src 类的指令先于 dst 类的指令完成。



需要插入 · 同 UB 地址有先后依赖

- 写后读：Store → Bar<VEC_STORE, VEC_LOAD> → Load
- 读后写：Load → Bar<VEC_LOAD, VEC_STORE> → Store
- 写后写：按最终写入顺序选择 VEC_STORE
- 跨循环轮次：后续轮次依赖前序 UB 结果时，也需要插入 Bar

无需插入 · 多插只会变慢

- 无前后依赖关系的指令
- 同名寄存器：可依赖寄存器保序机制
- 尽可能设计算法以减少/规避需要 Bar 的场景，提高指令并行度

```
// 正例 ✓ 写后读：插 barrier，Load 必须读取 Store 结果
StoreAlign(addr, sReg, mask);
LocalMemBar<VEC_STORE, VEC_LOAD>();
LoadAlign(lReg, addr);
// 反例 ✗ 缺少 barrier 时，Load 可能读到旧值。
StoreAlign(addr, sReg, mask);
LoadAlign(lReg, addr);
// 例外 ✓ 同名寄存器可由寄存器名保序。
AscendC::Reg::LoadAlign(workReg, xUbAddr + i * VL_B32);
AscendC::Reg::StoreAlign(xUbAddr + i * VL_B32, workReg, mask);
```

硬件资源/能力边界

寄存器资源

- **UnalignRegForLoad/Store** 各最多同时使用 4 个，超过会编译失败。
- 其他显式寄存器如 **RegTensor/MaskReg** 数量有限，编译器会尽量复用。超过不会失败，但溢出到 UB，性能下降。
- 建议控制编码使用的数量、缩短生命周期。

Aux Scalar 标量计算能力

支持:

- 不依赖循环状态的表达式，会被外提至 Main Scalar 计算
- 16/32 位整数的 “+”、“-”、“*”

不推荐: 16/32 位整数的 “÷”、“%” 依赖其他标量指令模拟

不支持: 非 16/32 位整数类型、浮点数类型

建议: 尽可能在 VF 外计算标量，通过入参传递进来

地址计算能力 —— 基于标量计算能力

支持: Aux Scalar 支持的所有标量计算

推荐: $\text{base} + c1*i + c2*j + c3*k + c4*t$ 这样的简单线性表达式，其中 $c1/c2/c3/c4$ 在循环内不变

不推荐:

- 复杂的地址表达式

数据访问能力 —— 和入参的限制呼应

支持:

- 访问 `__ubuf__` 上的数据
- 读写传入的标量参数

不支持:

- 访问非 `__ubuf__` 上的数据 (如 GM 空间)
- 访问成员变量等隐式状态

示例：标量计算/地址计算

SCALAR

// 推荐：外部预计算后传入

```
uint32_t scale = ((a + b) * 2U) / 4U;  
asc_vf_call<MyVF>(xUbAddr, yUbAddr, elemNum, vfLoopNum, scale);
```

// 不允许：循环内无法外提的浮点表达式

```
float scalar = (static_cast<float>(i) + a - b) * b / divisor; // i是循环变量  
float scalar2 = xUbAddr[0] + static_cast<float>(elemNum) / divisor;
```

ADDR

// 推荐：简单线性地址

```
uint32_t offset = i * VL_B32;  
LoadAlign(xReg, xUbAddr + offset);  
StoreAlign(yUbAddr + offset, xReg, mask);
```

// 不推荐：复杂的地址计算

```
uint32_t modOffset = static_cast<uint32_t>(i % 5U) * VL_B32; // i是循环变量  
uint32_t divOffset = static_cast<uint32_t>(i / 5U) * VL_B32; // i是循环变量
```

// 不允许：浮点计算

```
float offset = static_cast<float>(i) * static_cast<float>(VL_B32); // i是循环变量
```

第三章

SIMD VF 访存

GM→UB→寄存器→UB→GM 中的「UB↔寄存器」这一部分，按计算场景选指令

01

常规搬运 DIST_NORM

elemwise: 逐元素 1:1, 最常用

02

变位宽 UNPACK / PACK

半精度存储 ↔ 单精度计算

03

广播 BRC / 标量 FIRST_ELEMENT

标量铺满整条寄存器 / 归约写出

04

非连续 BLOCK_COPY / Gather

等间距跳写 / 按索引离散取数

05

特殊: 交织 / 掩码 / 非对齐

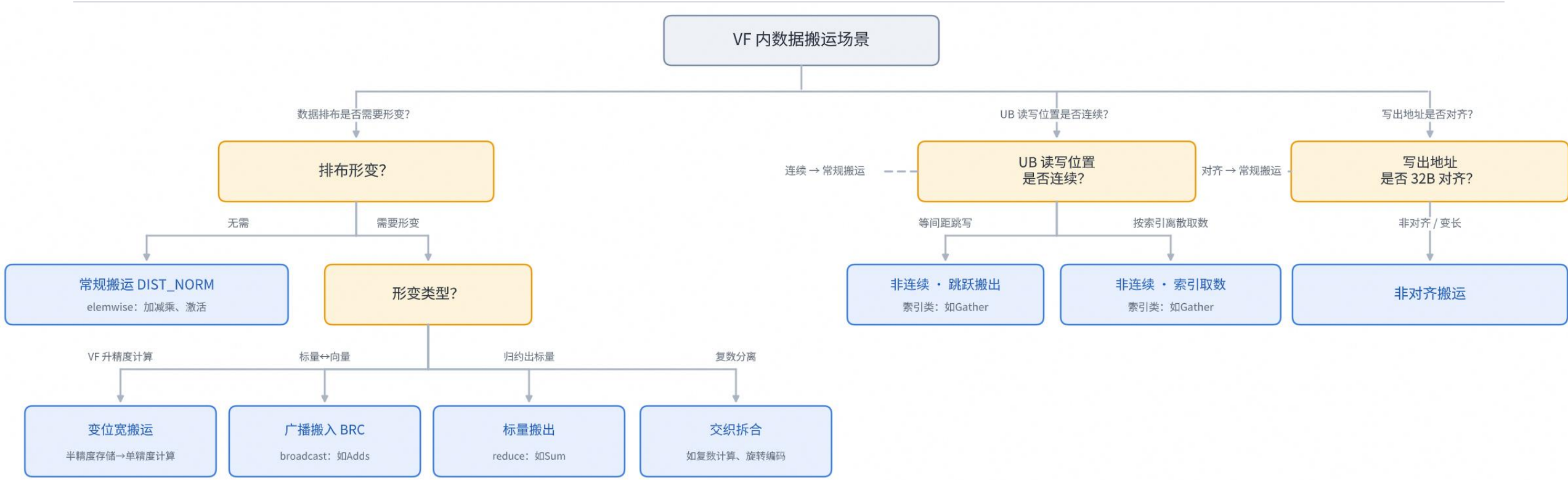
DINTLV·INTLV / MaskDist / UnAlign

06

寻址模式

NORM / POST_MODE_UPDATE / AddrReg

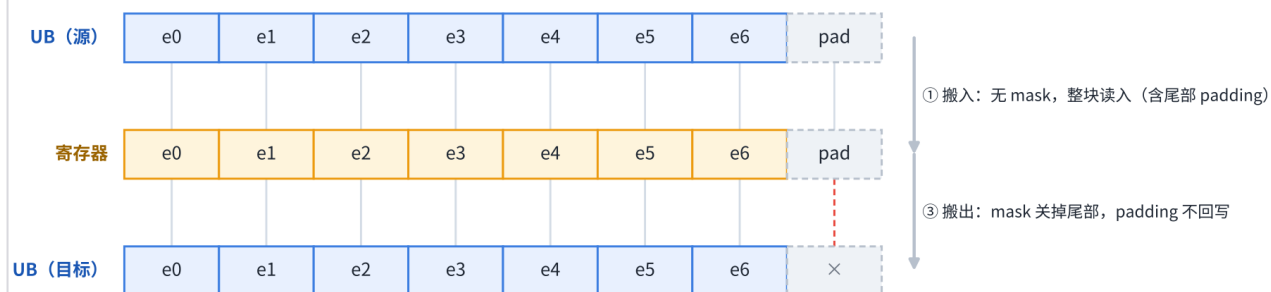
先看排布是否形变，再看连续与对齐



寻址模式（地址如何推进）和 mask（哪些 lane 生效）是与上述选择正交的两个维度，任何搬运方式都需配置

DIST_NORM: 逐元素 1:1, 搬入读全量、搬出按 mask

常规搬运 DIST_NORM: 逐元素 1:1, 搬入读全量、搬出按 mask



对应代码 (VF 循环体)

- ① LoadAlign<DIST_NORM>(vregA, xAddr); LoadAlign<DIST_NORM>(vregB, yAddr);
- ② Add(vregC, vregA, vregB, mask);
- ③ StoreAlign<DIST_NORM>(zAddr, vregC, mask);

若搬出不带 mask, 64 个 lane 全写回, 尾部垃圾会覆盖输出 buffer 的有效数据。

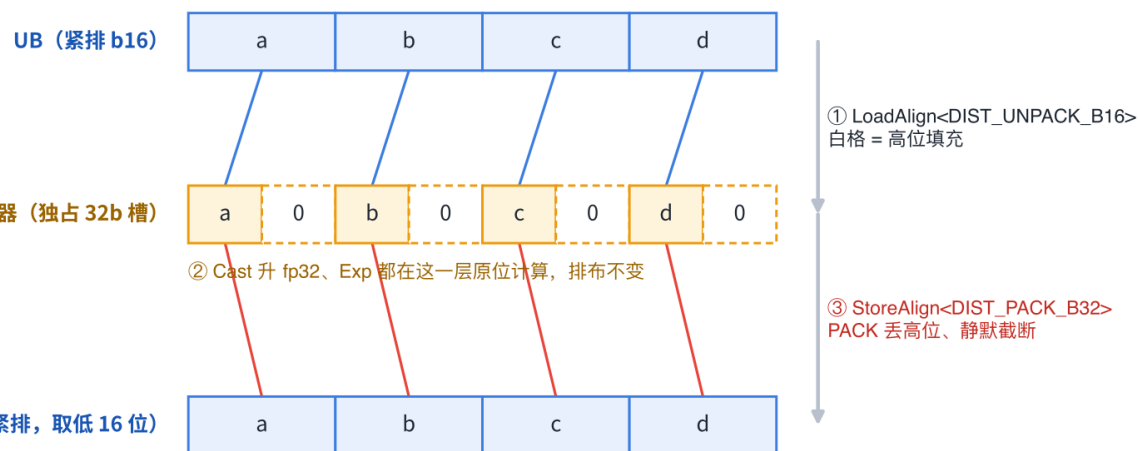
常规搬运 · 最常用

```
template <typename T>
__simd_vf__ void Scene1Vf(__ubuf__ T* x, __ubuf__ T* y,
    __ubuf__ T* z, uint32_t n, uint16_t loopNum) {
    uint32_t VL = AscendC::GetVecLen() / sizeof(T);
    RegTensor<T> a, b, c; MaskReg mask; uint32_t count = n;
    for (uint16_t i = 0; i < loopNum; i++) {
        mask = UpdateMask<T>(count);           // 收尾按 count 递减
        LoadAlign<T, DIST_NORM>(a, x + i*VL);   // 搬入无需 mask
        LoadAlign<T, DIST_NORM>(b, y + i*VL);
        Add(c, a, b, mask);
        StoreAlign<T, DIST_NORM>(z + i*VL, c, mask); // 搬出必带 mask
    }
}
// 反例 X 尾轮 mask 全开 → 垃圾 Lane 覆盖有效数据
```

源码 scene1_dist_norm.asc

UNPACK ↔ PACK: 半精度存储、单精度计算

升降位: UNPACK 紧排→间隔槽, PACK 取低 16 位压回 (中间 Cast/Exp 原位)



对应代码 (VF 循环体)

- ① LoadAlign<DIST_UNPACK_B16>(r16, xAddr);
- ② Cast<float>(r32, r16, m); Exp(r32, r32, m);
- ③ StoreAlign<DIST_PACK_B32>(yAddr, r16out, m);

mask 必须按宽类型 UpdateMask<float> 生成, 否则会去控制不存在的槽位。

变位宽 · 成对使用

```
template <typename T> // 例: bf16/fp16 存储, fp32 计算
__simd_vf__ void Scene2Vf(__ubuf__ T* x, __ubuf__ T* y,
    uint32_t n, uint16_t loopNum) {
    uint32_t VL = AscendC::GetVecLen() / sizeof(float);
    RegTensor<T> r16i, r16o; RegTensor<float> r32i, r32o;
    MaskReg mask; uint32_t count = n;
    for (uint16_t i = 0; i < loopNum; i++) {
        mask = UpdateMask<float>(count); // mask 跟「计算」宽类型走
        LoadAlign<T, DIST_UNPACK_B16>(r16i, x + i*VL); // b16 → 32 位宽
        Cast<float, T, CAST_B16_TO_B32>(r32i, r16i, mask);
        Exp(r32o, r32i, mask);
        Cast<T, float, CAST_B32_TO_B16>(r16o, r32o, mask);
        StoreAlign<T, DIST_PACK_B32>(y + i*VL, r16o, mask); // 压回 b16
    }
}
```

// 反例 X 用 UpdateMask<half> → 开出不存在的 Lane, 静默出错

源码 scene2_unpack_b16.asc · scene3_pack_b32.asc

BRC ↔ FIRST_ELEMENT: 标量铺满 / 归约写出

归约与广播：FIRST_ELEMENT 只写 output[i]（收）；BRC 把 1 个值铺满整批（放）



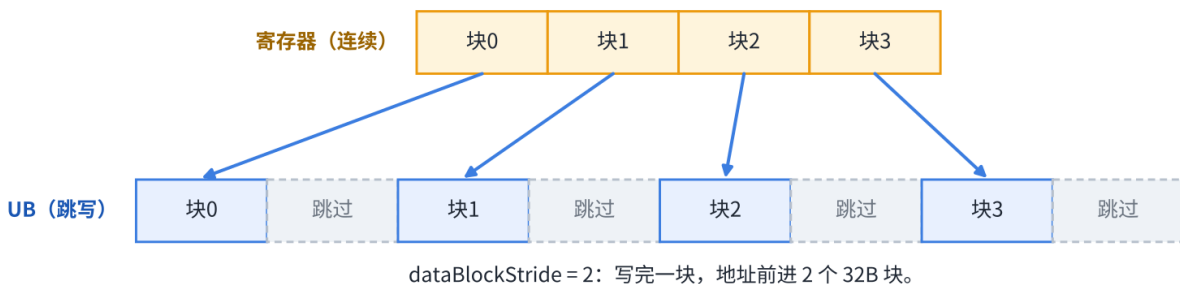
广播搬入 · 标量搬出

```
__simd_vf__ void Scene4Vf(__ubuf__ float* x, __ubuf__ float* y,
    uint32_t n, uint16_t loopNum) {
    uint32_t VL = AscendC::GetVecLen() / sizeof(float);
    RegTensor<float> in, sum; MaskReg mVL, mVL1;
    mVL1 = CreateMask<float, MaskPattern::VL1>(); // 只开第 0 lane
    uint32_t count = n;
    for (uint16_t i = 0; i < loopNum; i++) {
        mVL = UpdateMask<float>(count);
        LoadAlign<float, DIST_NORM>(in, x + i*VL);
        ReduceSum(sum, in, mVL); // 归约 → 首 lane
        StoreAlign<float, DIST_FIRST_ELEMENT_B32>(y + i, sum, mVL1);
        // 输出 +i 逐元素推进
    }
}
// 广播搬入: LoadAlign<float, DIST_BRC_B32>(bias, biasAddr); 1 标量铺满
// 反例 X 输出用 y + i*VL → 输出稀疏、归并读到零
```

源码 scene4_first_element.asc · scene5_brc_b32.asc

DATA_BLOCK_COPY: 按固定 stride 等间距跳写

DATA_BLOCK_COPY: 寄存器连续 → UB 按 32B 块跳写 (stride 单位 = 块)



对应代码 (VF 循环体)

- ① LoadAlign<DIST_NORM>(vreg, xAddr + i*VL);
- ② StoreAlign<DATA_BLOCK_COPY>(yAddr, vreg, blockStride, m);

单位是「块」不是字节 / 元素; 算错会静默写到错误位置。mask 仍按元素粒度截断。

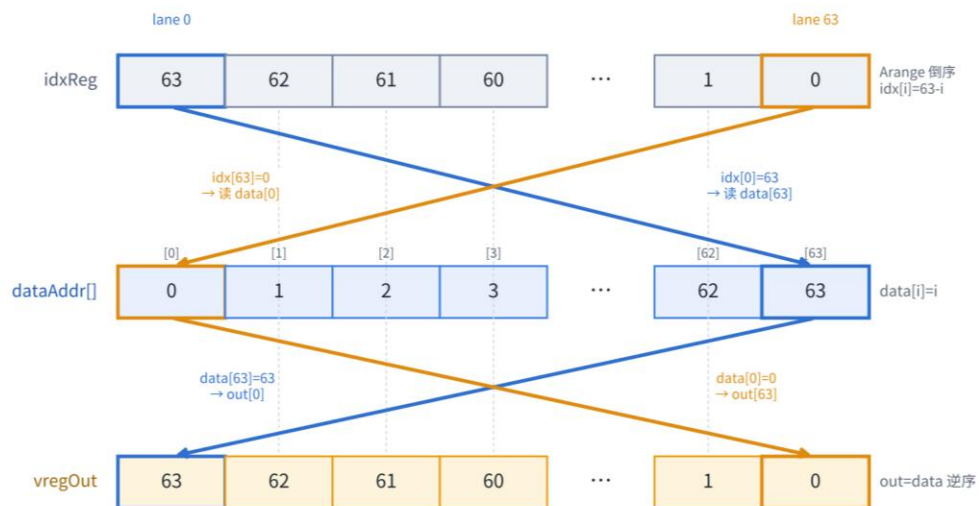
跳跃搬出

```
__simd_vf__ void Scene6Vf(__ubuf__ float* x,
    __ubuf__ float* y, __ubuf__ float* y2) {
    uint32_t VL = AscendC::GetVecLen() / sizeof(float);
    RegTensor<float> v; MaskReg mAll, mPart;
    uint32_t cntAll = VL, cntPart = 20;
    LoadAlign<float, DIST_NORM>(v, x);           // 连续搬入 64 元素
    mAll = UpdateMask<float>(cntAll);
    StoreAlign<float, DATA_BLOCK_COPY>(
        y, v, STRIDE, mAll);           // stride 单位=32B 块, 块间留空不覆盖
    mPart = UpdateMask<float>(cntPart);           // 20 Lane = 2.5 块
    StoreAlign<float, DATA_BLOCK_COPY>(y2, v, 1, mPart);
                                           // mask 逐元素粒度, 半块精确截断
}
// 反例 X stride 按字节/元素填 → 写到错误位置
```

源码 scene6_block_copy.asc

Gather: 逐 lane 按索引离散取数

Gather: 按索引取数 (Arange 直接生成索引) —— $out[i] = data[idx[i]]$



对应代码 (VF 循环体)

```
Arange<int32_t, DECREASE_ORDER>((RegTensor<int32_t>&)idxReg, 0); // idx[i]=63-i
Gather(vregOut, dataAddr, idxReg, preg); // out[i]=data[idx[i]]
StoreAlign<DIST_NORM>(outAddr, vregOut, preg);
```

逐 lane 独立寻址, 64 个输出互不重复 (无 E2B 的块内复制); 索引由 Arange 在寄存器内直接生成, 省去 UB 索引缓冲与 E2B 块展开。

按索引取数

```
__simd_vf__ void Scene7Vf(__ubuf__ float* data,
    __ubuf__ float* out) {
    uint32_t VL = AscendC::GetVecLen() / sizeof(uint32_t); // 64
    RegTensor<uint32_t> idx; RegTensor<float> v;
    MaskReg preg; uint32_t count = VL;
    preg = UpdateMask<uint32_t>(count);
    // 寄存器内直接生成倒序索引 idx[i]=63-i
    Arange<int32_t, IndexOrder::DECREASE_ORDER>(
        (RegTensor<int32_t>&)idx, 63);
    Gather<float, float, uint32_t>(v, data, idx, preg); // out=data[idx]
    StoreAlign<float, DIST_NORM>(out, v, preg); // 逐 lane 独立寻址
}
// 索引/位宽: b32/b64 配 u32, b8/b16 配 u16
// 反例 X 大张量 gather 用 16 位索引 → 索引溢出
```

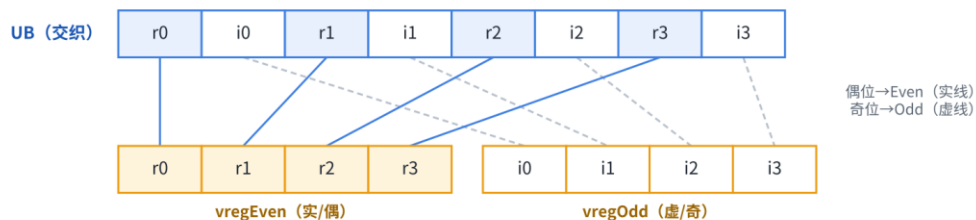
源码 scene7_gather_arange.asc

交织 / 掩码 / 非对齐：频次不高，按需查阅

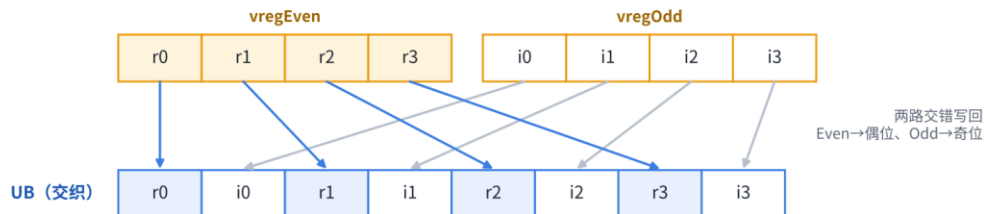
交织 DINTLV / INTLV

DINTLV 拆 / INTLV 合：交织 $r_i r_i \dots \rightleftharpoons$ 偶 $vregEven$ / 奇 $vregOdd$ (互为逆)

① 拆 DINTLV



③ 合 INTLV



对应代码 (VF 循环体)

```
① LoadAlign<DIST_DINTLV_B32>(vregEven, vregOdd, src);
② Abs(absE, vregEven, preg); Abs(absO, vregOdd, preg);
③ StoreAlign<DIST_INTLV_B32>(dst, absE, absO, preg);
```

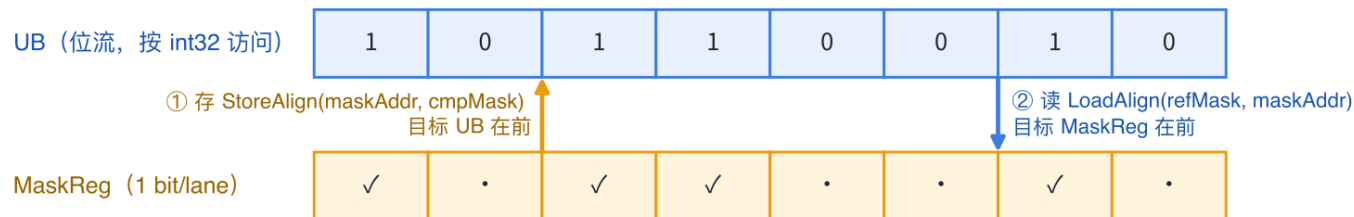
一次搬两个寄存器、步长 $\times 2$ ；拆开各自计算、算完再交错合并写回。

- 复数 $[r, i]$ 按奇偶拆成两寄存器，各自算完再交错合并。
- 双寄存器搬运数据量 $\times 2$ ，步长按 $VL \times 2$ 推进，一份 mask 作用两寄存器。

交织 / 掩码 / 非对齐：频次不高，按需查阅

掩码存取 MaskDist

MaskDist: MaskReg 位流 $\leftrightarrow$ UB (整型位流访问, 固定 256 bit)



对应代码 (VF 循环体)

```
① StoreAlign<MaskDist::DIST_NORM>(maskAddr, cmpMask);  
② LoadAlign<MaskDist::DIST_NORM>(refMask, maskAddr);  
③ MaskAnd(m, m, refMask, all); Select(out, zeros, data, m);
```

目标在前、源在后；按数据类型解释位流无意义；同地址存→读需插 LocalMemBar；US/DS 采样时 mask 同步采样。

- MaskReg $\leftrightarrow$ UB 暂存 (固定占 32B / 256bit) , 存读参数顺序相反。
- 同址先存后读需要插 LocalMemBar<VEC_STORE, VEC_LOAD> , 否则读回不确定。

交织 / 掩码 / 非对齐：频次不高，按需查阅

非对齐 StoreUnAlign + Post

非对齐三件套：① 声明暂存 → ② 循环累积 → ③ Post 收尾（漏 ③ 就丢尾）



- 流程：声明 UnalignReg → 循环内累积 → 循环外 Post 刷尾。
- POST_MODE_UPDATE 自动推进地址；漏掉 Post 则尾部残余丢失。

正交维度 & 误区

寻址模式+ 常见误区

POST_MODE_NORMAL 偏移在调用点手动算 + i*VL	默认	POST_MODE_UPDATE 地址按引用线性前进——别再手动加偏移	AddrReg 多维 / gather 偏移，多占一个地址寄存器
---	-----------	---	--

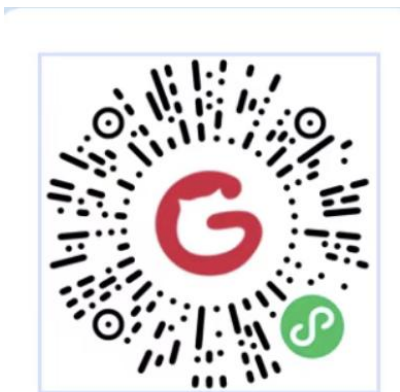
常见误区	后果	正解
搬出尾轮 mask 全开	无效 lane 覆盖有效数据	<code>UpdateMask<T>(count)</code> 收尾
升降位按存储类型生成 mask	开出不存在的 lane	mask 跟计算类型 <code>UpdateMask<float></code>
<code>dataBlockStride</code> 按字节/元素填	写到错误位置	单位是 32B 块
<code>POST_MODE_UPDATE</code> 后又手动加偏移	地址双重推进、错位	UPDATE 与手动偏移二选一
同址存 mask 后立即读回	概率性读到全 0	<code>LocalMemBar<VEC_STORE, VEC_LOAD></code>
对齐搬入地址非 32B 对齐	设备异常 507035	mask / UnAlign 收尾变长

Thank you.

社区愿景：打造开放易用、技术领先的 AI 算力新生态。

社区使命：使能开发者基于 CANN 社区自主研究创新，构筑共享共赢的 CANN 生态。

<https://gitcode.com/cann/cann-samples>



扫码报名
CANN开发者Meetup
6月27日 上海站



上 CANN 社区获取干货



关注 CANN 公众号获取资讯